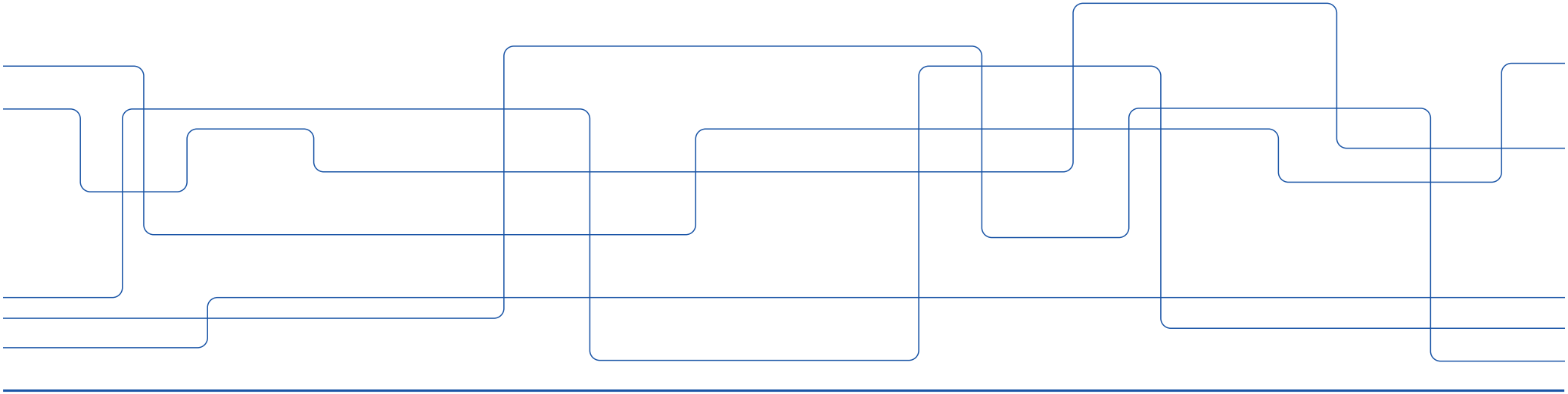


HI1024

Grundkurs i programmering

F10: Algoritmer I!



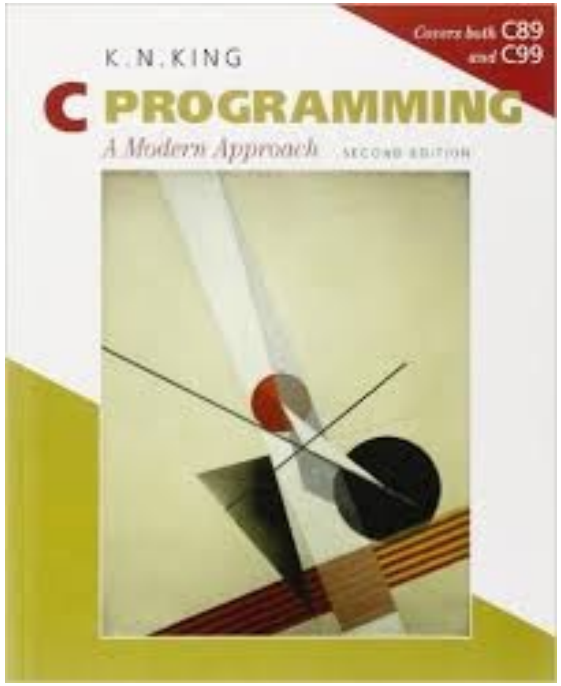
Innehåll!

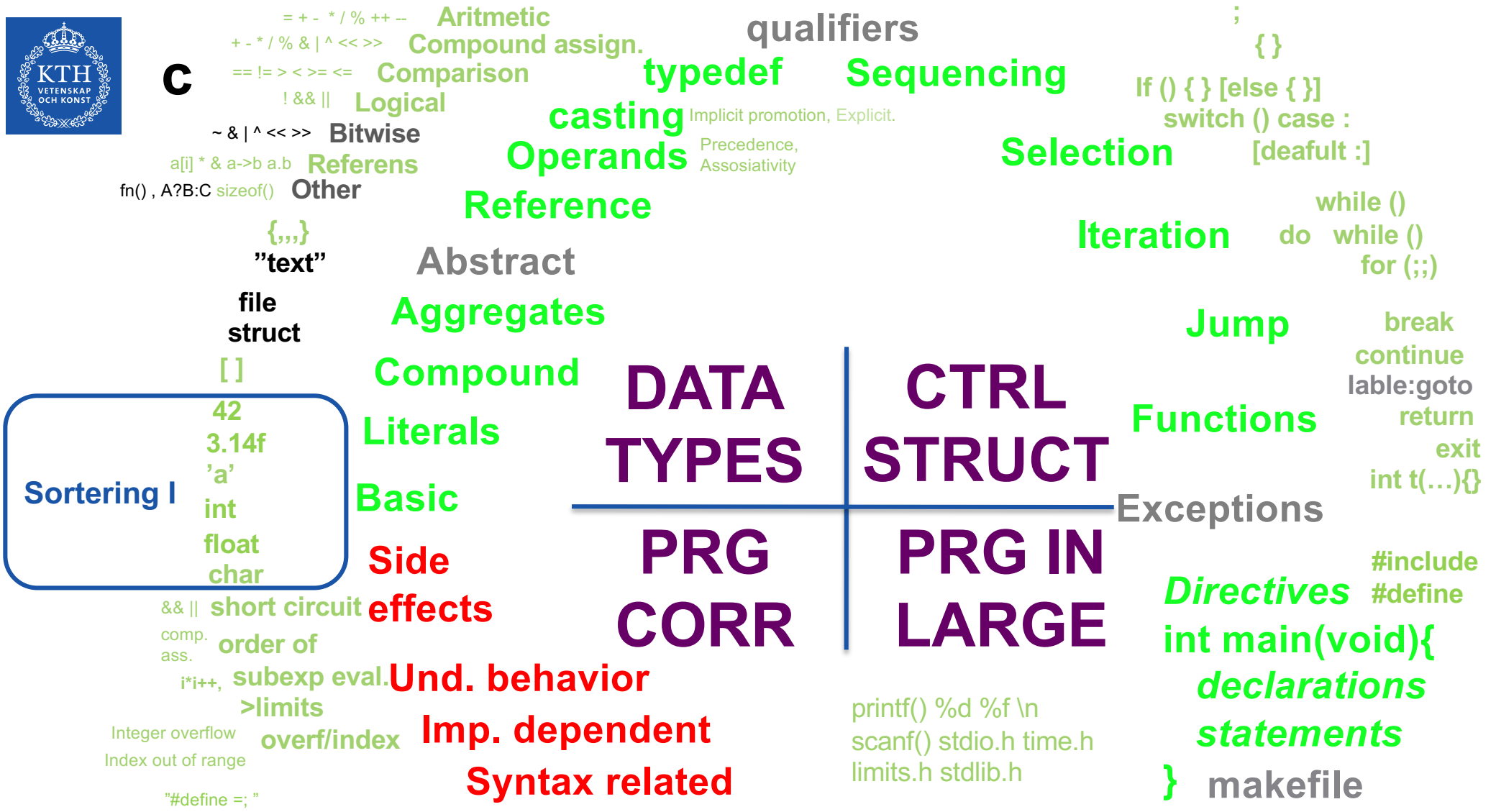
King sida 205..8

Repetition föreläsning #9

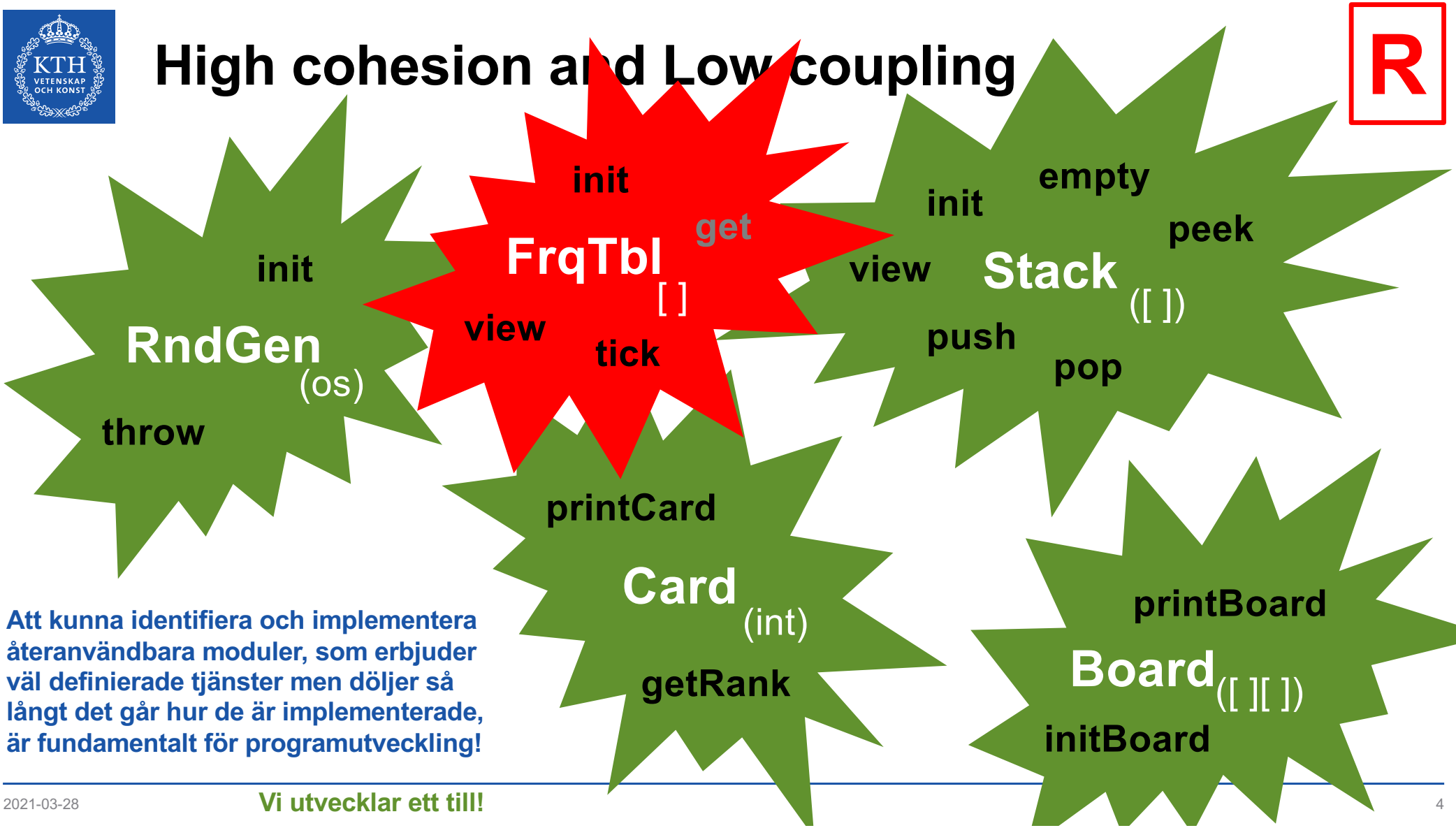
Sortering

Övning




C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
Aggregates
Compound
Literals
Basic
42
3.14f
’a’
int
float
char
Sortering I
Abstract
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence, Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
Iteration
while ()
do while ()
for (;;)
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
}
makefile
printf() %d %f \n
scanf() stdio.h time.h
limits.h stdlib.h
&& || short circuit
comp. ass.
order of
i*i++,
subexp eval.
>limits
Integer overflow
overf/index
Index out of range
”#define =; ”
Side effects
Und. behavior
Imp. dependent
Syntax related
int/int
/*comment*/ or //

High cohesion and Low coupling



Att kunna identifiera och implementera återanvändbara moduler, som erbjuder väl definierade tjänster men döljer så långt det går hur de är implementerade, är fundamentalt för programutveckling!

Pekare

```c
#include <stdio.h>

int main(void){
    char ch = 'A';
    char *pCh = &ch;  // & Ge mig adressen till…

    printf("Pointer tester 1.0\n");

    printf("Value of ch = %c\n", ch);

    *pCh = 'B';       // * Innehållet i adressen…

    printf("Value of ch = %c\n", ch);

    printf("Pointer tester 1.0 Done\n");
}
```

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
   *
   *
 VART              VÄRDE NAMN TYP
#7FFF 55F1 AC1C:'A''B'ch   char
#7FFF 55F1 AC1D:7…1C  pCh char*
#7FFF 55F1 AC1E:
#7FFF 55F1 AC1F:
#7FFF 55F1 AC20:

Pointer tester 1.0
Value of ch = A
Value of ch = B
Pointer tester 1.0 Done
```

R

Wow: & ger adressen till en variabel, och * innehållet i en adress

Hum, vad händer om vi använder en adress som ett funktionsargument?

Pekare

(Funktionen tar emot ett heltal)

```c
#include <stdio.h>

void times2(int number) {
   printf("...double in  :%d\n",number);
   number=number*2;
   printf("...double out :%d\n",number);
}

int main(void){
  int nb=7;

  printf("Functions & pointers!\n");
  printf("Before call   :%d\n",nb);
  times2(nb);
  printf("After call    :%d\n",nb);

  return 0;
}
```

(Vi skickar en kopia av nb:s värde)

(Funktionen tar emot en adress)

```c
#include <stdio.h>

void times2(int *nbPtr) {
   printf("...double in  :%d\n",*nbPtr);
   *nbPtr=*nbPtr*2;
   printf("...double out :%d\n",*nbPtr);
}

int main(void){
  int nb=7;

  printf("Functions & pointers!\n");
  printf("Before call   :%d\n",nb);
  times2(&nb);
  printf("After call    :%d\n",nb);

  return 0;
}
```

(Vi skickar vart nb finns i minnet)

Vad händer nu om vi skickar adressen till vart argumentet finns i stället?

Pekare

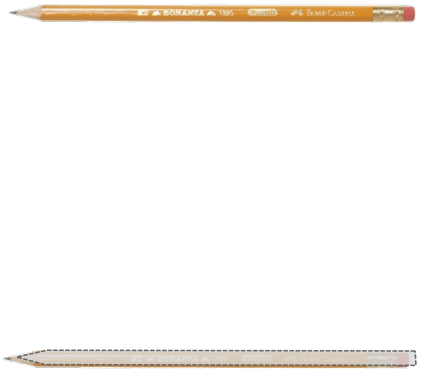
```c
#include <stdio.h>

void times2(int *nbPtr) {
   printf("...double in  :%d\n",*nbPtr);
   *nbPtr=*nbPtr*2;
   printf("...double out :%d\n",*nbPtr);
}

int main(void){
  int nb=7;

  printf("Functions & pointers!\n");
  printf("Before call   :%d\n",nb);
  times2(&nb);
  printf("After call    :%d\n",nb);

  return 0;
}
```



```
PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
   *
   *
 VART    VÄRDE      NAMN      TYP
#7…1C:    7 14       nb       int
#7…1D:   #7…1C     nbPtr      int*
#7…1E:
#7…1F:
#7…20:
```

R

```
Functions & pointers!
Before call   :7
...double in  :7
...double out :14
After call    :14
```


…Wow, raden dubblerar innehållet i den utpekade minnescellen, som är anropande funktions argument!

Pekare

```c
#include <stdio.h>

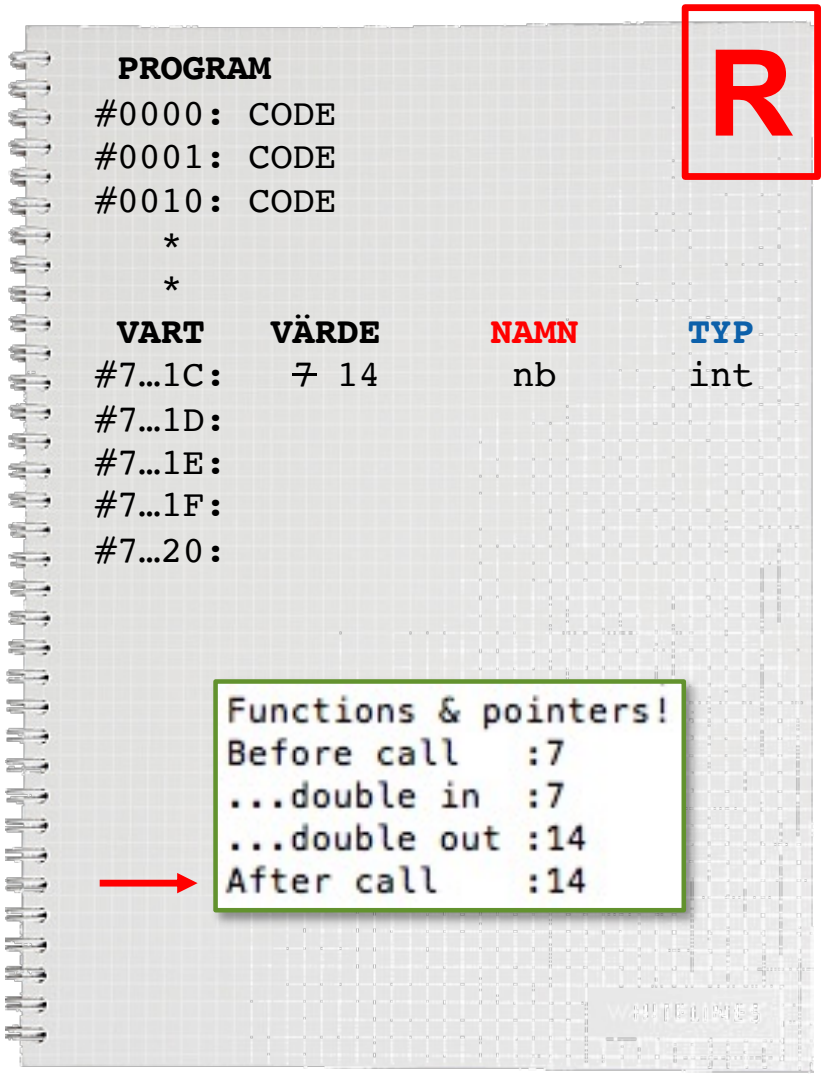
void times2(int *nbPtr) {
   printf("...double in  :%d\n",*nbPtr);
   *nbPtr=*nbPtr*2;
   printf("...double out :%d\n",*nbPtr);
}

int main(void){
  int nb=7;

  printf("Functions & pointers!\n");
  printf("Before call    :%d\n",nb);
  times2(&nb);
  printf("After call     :%d\n",nb);

  return 0;
}
```





…åter i den anropande funktionens kod så har argumentet värde ändrats magiskt!



Sammantaget



1. En ej initierad pekarvariabel är ett slumptal, som kan avsluta programmet om pekaren används (Segmentation Fault)!
2. Makrot **NULL** kan användas för att ge en pekare ett värde som betyder att pekaren för stunden inte pekar på något!
3. Flera pekarvariabler kan peka på samma variabel!
4. Glöm aldrig & eller * där de krävs, det blir alltid svåra fel!
5. **Variabelnamnet på en matris (utan [ ]) kan användas som en pekare till första elementet, det är därför & inte behövs i funktionsanrop**.
6. Döp alltid pekarvariabler med prefixet p (eller möjligen postfix Pointer alt Ptr).
7. Innebörden av *&r1 är r1, d.v.s r1:s värde, ty tolkas *(&r1)!



Varför lär vi oss pekare?



1. **Pekare möjliggör att funktioner kan ändra värdet på anropande funktionsargument. Helt avgörande kunskap för att klara denna kurs! (Utan denna egenskap kan en funktion bara returnera ett värde.)**
2. **Eftersom c kopierar funktionsargument blir funktionsanropet mycket snabbare om en pekare till argumentet används! Vi kommer att se det senare i denna kurs!**
3. Pekare möjliggör att ett program kan skapa variabler när de behövs dynamiskt, inte enbart statiskt via deklarationer. Helt avgörande i de flesta reella programutvecklingsprojekt, får dock vänta till senare kurser.
4. Pekare möjliggör väldigt användbara speciella dynamiska datastrukturer som kallas länkade listor & träd. Helt avgörande i de flesta reella programutvecklingsprojekt, får dock vänta till senare kurser.

printf() – din bästa vän!

```c
#include <stdio.h>
#define STACKSIZE 10
#define DBGLEVEL 2                          // 2..0 Most verbose to none

//*** Debug package ********************************************
void dbgEnter(char *msg) {
    switch (DBGLEVEL) {
        case 2: printf(">>> %s\n",msg); //break intensional omitted
    }
}
void dbgExit(char *msg) {
    switch (DBGLEVEL) {
        case 2: printf("<<< %s\n",msg); //break intensional omitted
    }
}
void dbgInspect(char *msg, char *id, int value) {
    switch (DBGLEVEL) {
        case 1:                             //break intensional omitted
        case 2: printf("--- %s: %s=%d\n",
                       msg, id, value); //break intensional omitted
    }
}
//*** DBGP END *************************************************
void initStack(int aStack[]){
    dbgEnter("initStack");
    aStack[0]=0;
    dbgExit("initStack");
}
void printStack(int aStack[]){
    dbgEnter("printStack");
    dbgInspect("printStack", "items", aStack[0]);
```

```
RPN Calculator 1.0
>>> initStack
<<< initStack
3 2 1 + + =
>>> printStack
--- printStack: items=1
[ 3 ]
<<< printStack
>>> printStack
--- printStack: items=2
[ 2 3 ]
<<< printStack
>>> printStack
--- printStack: items=3
[ 1 2 3 ]
<<< printStack
>>> printStack
--- printStack: items=2
[ 3 3 ]
<<< printStack
>>> printStack
--- printStack: items=1
[ 6 ]
<<< printStack
>>> printStack
--- printStack: items=0
[ ]
<<< printStack
RPNC Done, bye!
```

```
RPN Calculator 1.0
3 2 1 + + =
--- printStack: items=1
[ 3 ]
--- printStack: items=2
[ 2 3 ]
--- printStack: items=3
[ 1 2 3 ]
--- printStack: items=2
[ 3 3 ]
--- printStack: items=1
[ 6 ]
--- printStack: items=0
[ ]
RPNC Done, bye!
```

```
RPN Calculator 1.0
3 2 1 + + =
[ 3 ]
[ 2 3 ]
[ 1 2 3 ]
[ 3 3 ]
[ 6 ]
[ ]
RPNC Done, bye!
```

2 1 0

DBGLEVEL

Nytt: Algoritmer och datastrukturer

- En **algoritm** är, inom matematiken och datavetenskapen, ändlig uppsättning (mängd) otvetydiga instruktioner som efter exekvering löser ett problem.[1] Algoritmen startar i ett givet tillstånd (starttillstånd) och når resultatet (sluttillstånd) inom ett ändligt antal steg. Varje steg måste var tydligt och precist definierat, på så sätt att utomstående ska kunna exekvera algoritmen och verifiera ett resultat. Ytterligare är *effektivitet* viktigt, det vill säga varje steg måste vara elementärt och exakt, samt gå att beräkna inom en ändlig tidsram.

Mätdata…

13
13,5
14
13
13,5
13
14

…algoritm…

Medelvärde
Typvärde
Max
Median
Sortera
Min

…datastruktur!

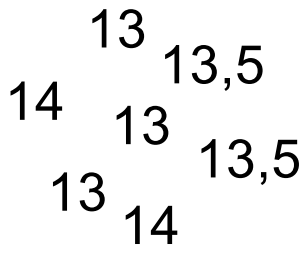
Matris
Lista
Träd
Hashtabell
Stack (LIFO)
Kö (FIFO)

Du måste känna till en del algoritmer, och passande datastrukturer så att algoritmerna kan utföras effektivt!

Nytt: Algoritmer och datastrukturer

- Och som vanligt vill vi definitivt dölja bort algoritmens datastruktur, men kanske också algoritmen, så att ett antal väl valda funktioner kan leverera efterfrågade tjänster utan att exponera något annat...
- …i Lab #2 är tanken att ett litet väl valt antal funktioner ska erbjuda statiska funktioner så som min, max & medelvärde utan att den som använder tjänsterna ska behöva sätta sig in i hur de faktiskt är realiserade!

Mätdata…



…algoritm…

Medelvärde
Typvärde
Max
Median
Sortera
Min

…datastruktur!

Matris
Lista
Träd
Hashtabell
Stack (LIFO)
Kö (FIFO)

Det är dags att lära sig några algoritmer som kan sortera tal (och bokstäver)!



Sortering: Selection Sort

Sortering är ett väldigt vanligt problem, därför finns det redan många vedertagna algoritmer med lite olika fokus för detta!

När ni går ut så kommer ni att känna igen ett flertal, men i denna kurs så skrapar vi bara lite på ytan…

Vi börjar enkelt: Hitta det största talet i en matris och placera det sist i matrisen (om det inte redan ligger där). Talet som låg sist placerar vi där vi hämtade det största talet!

T.ex: 12 3 6 **18** 7 15 **10** ska bli 12 3 6 **10** 7 15 **18**

Sortering: Selection Sort

```c
#include <stdio.h>
#define SIZE 7

void viewArr(int anArr[], int size) {
    printf("[ ");
    for (int i=0; i<size; i++)
        printf("%d ", anArr[i]);
    printf("]\n");
}

int main(int argc, char **argv) {
    int data[SIZE]={12,3,6,18,7,15,10};
    printf("Selection Sort Test!\n");

    viewArr(data,SIZE);

    printf("SST Done, bye!\n");
    return 0;
}
```



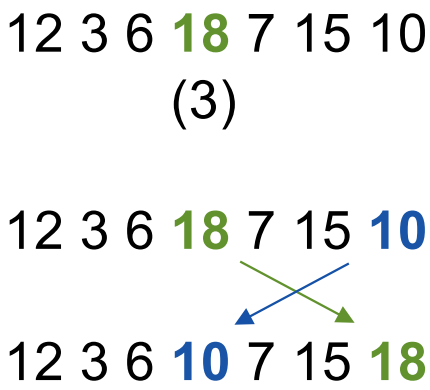
En godtycklig slumpsekvens som vi testar med!

```
Selection Sort Test!
[ 12 3 6 18 7 15 10 ]
SST Done, bye!
```


Som alltid, skapar vi först objektet (initierat) och ett sätt att skriva ut det – när det fungerar är det dags att sortera!

Sortering: Selection Sort

```
// sortArr(int anArr[], int size) {
//
//
//
//
//
//
//
//
}
```



Hur ser den här beskrivningen ut i c-kod?

Sortering: Selection Sort

```
// sortArr(int anArr[], int size) {
//   int at=0, tmp

//   Gå igenom alla tal…
//       Större än tidigare största?
//           Ja! Spara positionen

//   Byt plats på innehållet i
//   positionen at med det som
//   står sist i arrayen!
}
```

12 3 6 **18** 7 15 10

(3)

12 3 6 **18** 7 15 **10**



12 3 6 **10** 7 15 **18**

Hur ser den här beskrivningen ut i c-kod?

Sortering: Selection Sort

```
Selection Sort Test!
[ 12 3 6 18 7 15 10 ]
[ 12 3 6 10 7 15 18 ]
SST Done, bye!
```

```
// sortArr(int anArr[], int size) {
//   int at=0, tmp

//   Gå igenom alla tal…
//       Större än tidigare största?
//           Ja! Spara positionen

//   Byt plats på innehållet i
//   positionen at med det som
//   står sist i arrayen!
}
```

```
void sortArr(int anArr[], int size) {
    int at=0,tmp;

    for (int i=1; i<size; i++)
        if (anArr[i]>anArr[at])
            at=i;

    tmp=anArr[at];
    anArr[at]=anArr[size-1];
    anArr[size-1]=tmp;
}
```

Sortering: Selection Sort

```
Selection Sort Test!
[ 12 3 6 18 7 15 10 ]
[ 12 3 6 10 7 15 18 ]
SST Done, bye!
```

12 3 6 **10** 7 15 **18**
12 3 6 10 7 **15** 18
7 3 6 10 **12** 15 18
7 3 6 **10** 12 15 18
6 3 **7** 10 12 15 18
3 **6** 7 10 12 15 18

Hum… …det högsta talet är nu bergis sist.
Vad händer om jag anropar sort igen med samma matris utom sista värdet?!

…efter 1:a anropet
…efter 2:a anropet
…efter 3:e anropet
…efter 4:e anropet
…efter 5:e anropet
…efter 6:e anropet, wow helt rätt!

Sortering: Selection Sort

```
Selection Sort Test!
[ 12 3 6 18 7 15 10 ]
[ 12 3 6 10 7 15 18 ]
SST Done, bye!
```

12 3 6 **10** 7 15 **18**
12 3 6 10 7 **15** 18
7 3 6 10 **12** 15 18
7 3 6 **10** 12 15 18
6 3 **7** 10 12 15 18
3 **6** 7 10 12 15 18

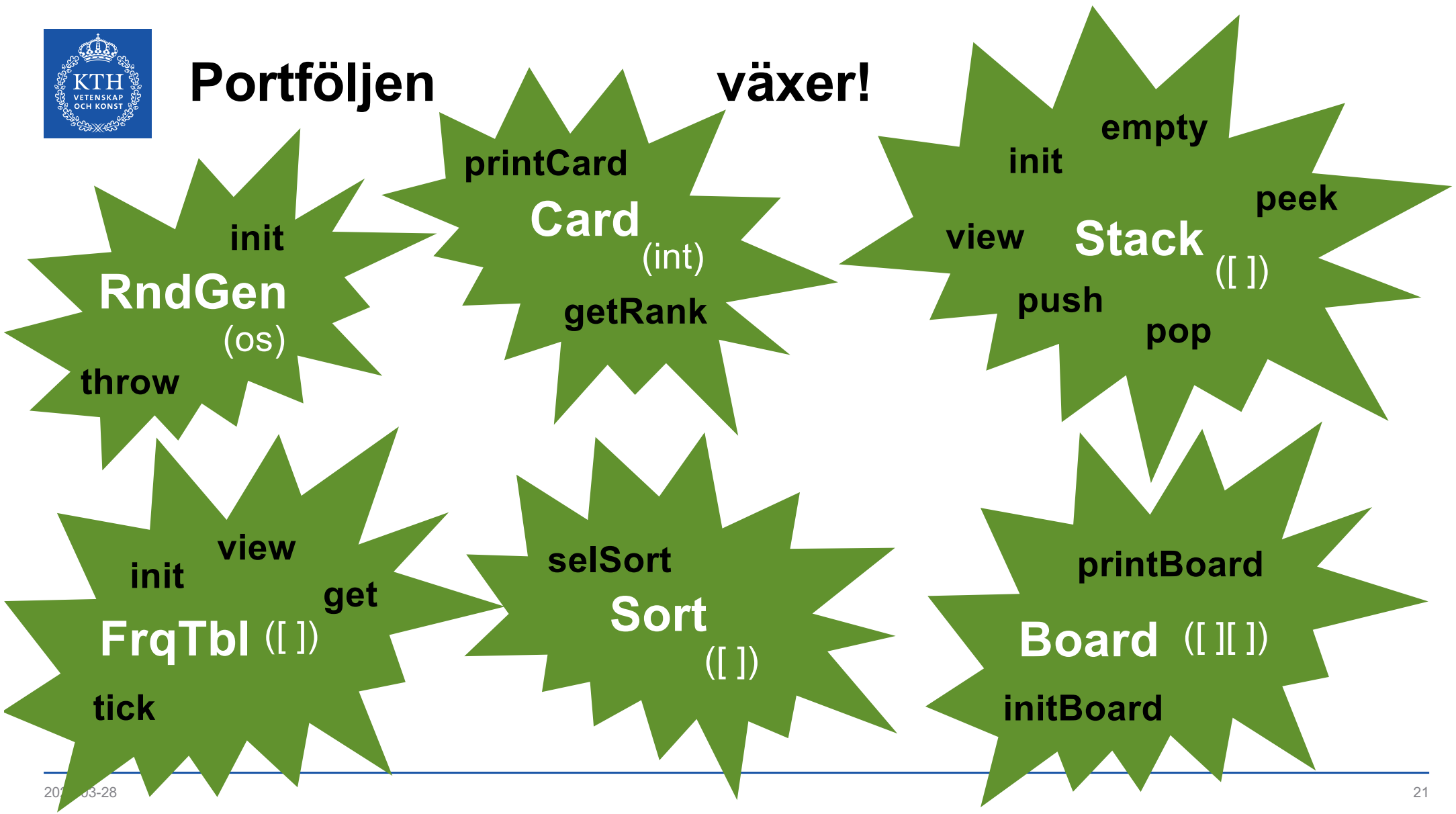
```
Selection Sort Test!
[ 12 3 6 18 7 15 10 ]
[ 3 6 7 10 12 15 18 ]
SST Done, bye!
```

```
void sortArr(int anArr[], int size) {
    int at=0,tmp;

    for (int i=1; i<size; i++)
        if (anArr[i]>anArr[at])
            at=i;

    tmp=anArr[at];
    anArr[at]=anArr[size-1];
    anArr[size-1]=tmp;
    if (size>1) sortArr(anArr,size-1);
}
```

Ett rekursivt anrop! **(går också utan rekursion:for/for)**


Portföljen
växer!
RndGen
(os)
init
throw
Card
(int)
printCard
getRank
Stack
([ ])
empty
init
peek
view
push
pop
FrqTbl
([ ])
view
init
get
tick
Sort
([ ])
selSort
Board
([ ][ ])
printBoard
initBoard



Typvärde…

Mätdata…

13 13,5
14 13 13,5
13 14

…algoritm…

Medelvärde

Typvärde

Max Median

Sortera **Min**

…datastruktur!

Matris

Lista

Träd Hashtabell

Stack (LIFO)

Kö (FIFO)

Kursens examinator är väldigt förtjust i frekvenstabeller & statistik…
…frekvenstabeller, medelvärde, median eller typvärde finns ofta på TEN2…
…frekvenstabeller kan vi, medelvärde också, men hur får vi fram typvärdet?

(Typvärdet är det tal som förekommer flest gånger i serien.)



Typvärde…

Mätdata…

13 13,5 14 13 13,5 13 14

…algoritm…

Medelvärde

Typvärde

Max Median

Sortera **Min**

…datastruktur!

Matris

Lista

Träd

Hashtabell

Stack (LIFO)

Kö (FIFO)

Kursens examinator är väldigt förtjust i frekvenstabeller & statistik…
…frekvenstabeller, medelvärde, median eller typvärde finns ofta på TEN2…
…frekvenstabeller kan vi, medelvärde också, men hur får vi fram typvärdet?

(Typvärdet är det tal som förekommer flest gånger i serien.)

Ett sätt, om talen är sorterade, så är det det tal som förekommer i den längsta egna sekvensen!



Typvärde…

```
Selection Sort Test!
[ 7 12 3 6 12 7 18 7 15 10 ]
[ 3 6 7 7 7 10 12 12 15 18 ]
Type value is 7
SST Done, bye!
```

```
// typvärde anArr[] size {
//
//
//
//
//
//
//
//
//
//
//
//
//
//   }
```



Typvärde…

```
Selection Sort Test!
[ 7 12 3 6 12 7 18 7 15 10 ]
[ 3 6 7 7 7 10 12 12 15 18 ]
Type value is 7
SST Done, bye!
```

```
// typvärde anArr[] size {
//   int at=0, max=1, cnt=1;
//   För alla tal från 2:a…
//      om samma som föregående
//         cnt++
//      i annat fall
//         om cnt>max  (nytt max)
//            max=cnt
//            at=current-1
//
//         cnt=1   (börja på ny sek)
//
//   returnera anArr[at]
//   }
```

Typvärde…

```
Selection Sort Test!
[ 7 12 3 6 12 7 18 7 15 10 ]
[ 3 6 7 7 7 10 12 12 15 18 ]
Type value is 7
SST Done, bye!
```

```
// typvärde anArr[] size {
//   int at=0, max=1, cnt=1;
//   För alla tal från 2:a…
//     om samma som föregående
//       cnt++
//     i annat fall
//       om cnt>max  (nytt max)
//           max=cnt
//           at=current-1
//
//       cnt=1   (börja på ny sek)
//
//   returnera anArr[at]
//   }
```

```
int typeValue(int anArr[], int size) {
    int at=0, max=1, cnt=1;
    for (int i=1; i<size; i++)
        if (anArr[i]==anArr[i-1])
            cnt++;
        else {
            if (cnt>max) {
                max=cnt;
                at=i-1;
            }
            cnt=0;
        }
    return anArr[at];
}
```



Bubble sort

Selection sort är ett mycket ineffektiv sätt att sortera (men lättbegriplig)…
…för algoritmen ”fixar till” så lite för varje iteration genom matrisen!

Kunde algoritmen gör mer än att ”bara” ”fixa till” ”sista talet”…
…skulle det kanske krävas färre operationer för att bli klar!

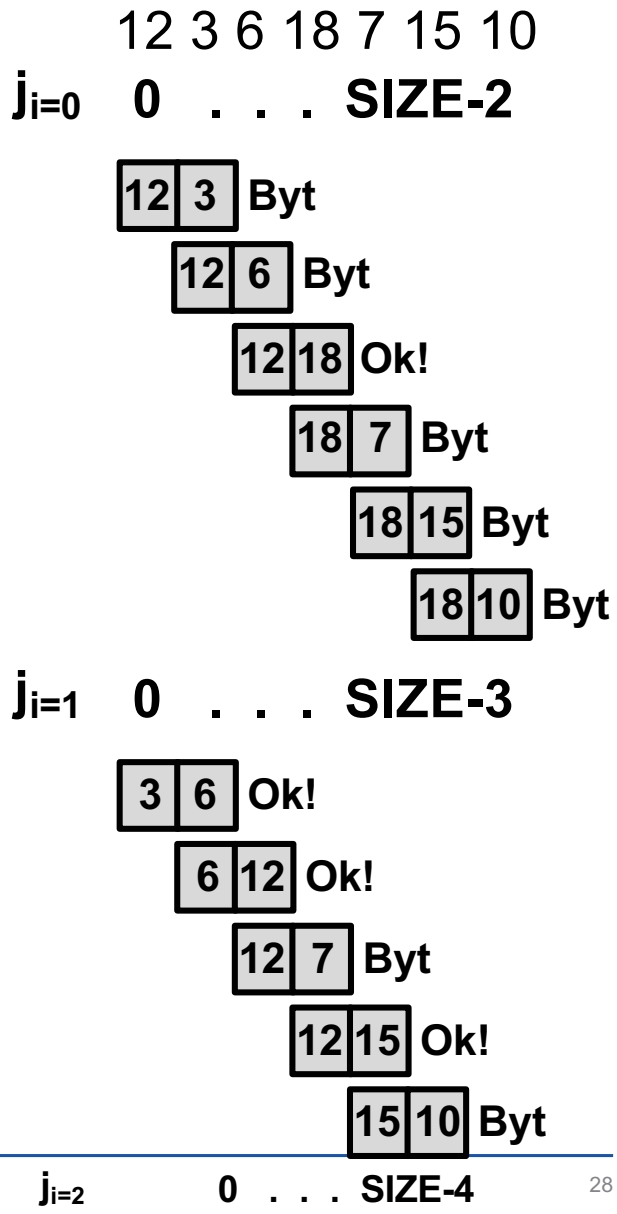
En känd metod är att gå igenom hela matrisen och byta plats på närliggande tal om de är i fel ordning. Det största talet kommer bergis att hamna rätt första gången, men flera andra tal kan också ha flyttats till rätt position, eller i alla fall en bit mot rätt position. Därefter upprepas proceduren på matrisen utom sista talet på samma sätt som selection sort.

Alg. kallas för **Bubble sort** för felplacerade tal ”bubblar” upp mot rätt plats!

Bubble sort

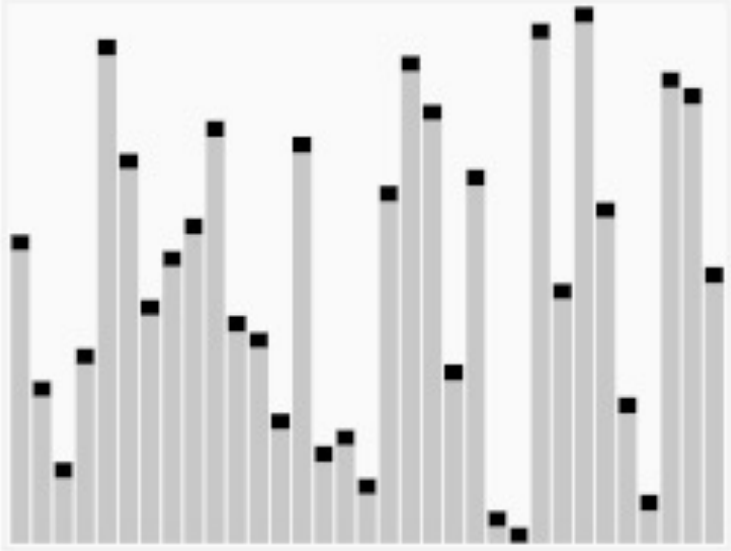
```
void sortArr(int anArr[], int size) {
    int tmp;

    for (int i=0; i<size-1; i++)
        for (int j=0; j<size-1-i; j++)
            if (anArr[j]>anArr[j+1]) {
                tmp=anArr[j];
                anArr[j]=anArr[j+1];
                anArr[j+1]=tmp;
            }
}
```

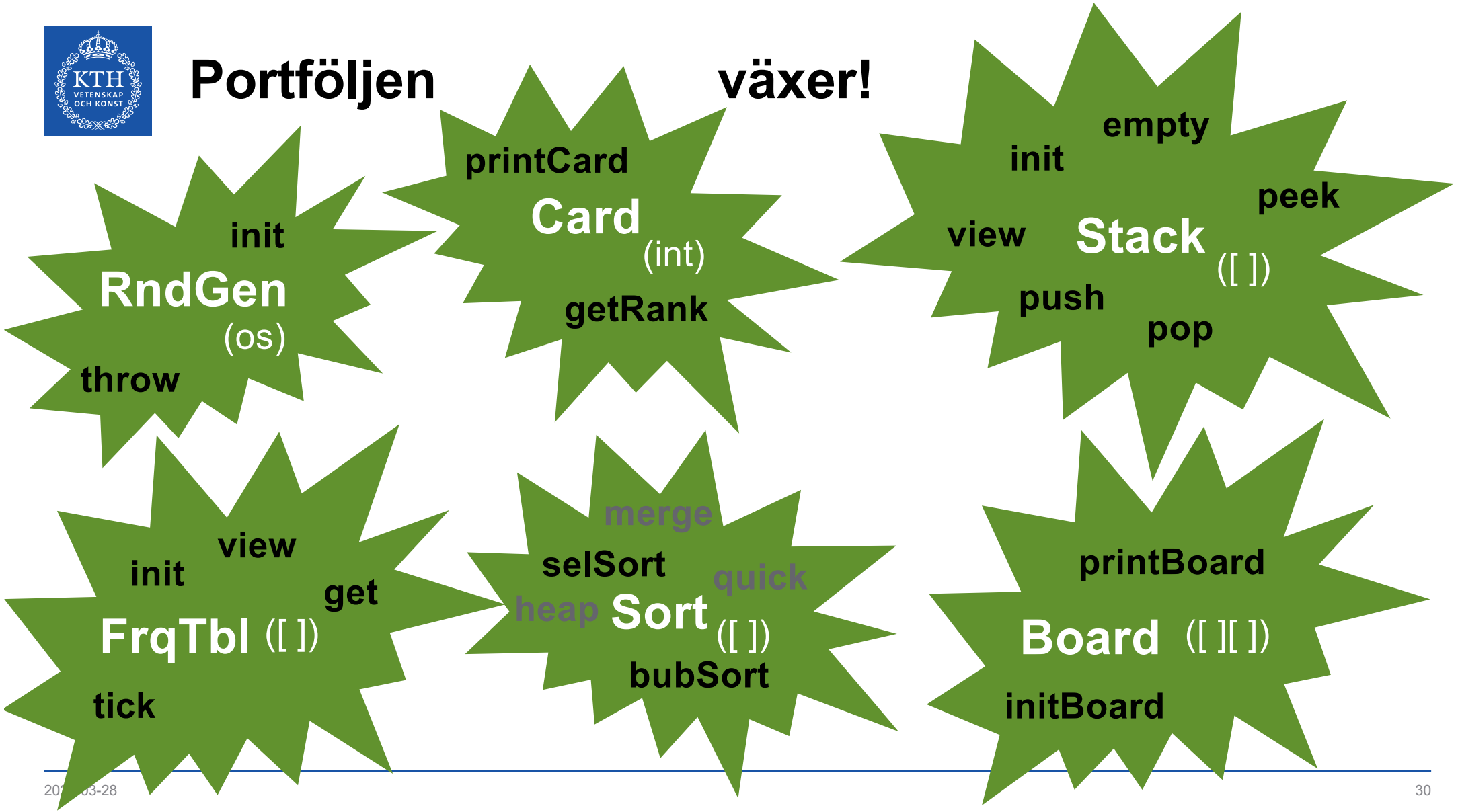


Algodat-kursen: Quick/Merge/Heap Sort etc

- Quicksort (sometimes called partition-exchange sort) is an efficient sorting algorithm, serving as a systematic method for placing the elements of an array in order. Developed by Tony Hoare in 1959,[1] with his work published in 1961,[2] it is still a commonly used algorithm for sorting. When implemented well, it can be about two or three times faster than its main competitors, **merge sort** and **heap sort**.[3]



Portföljen växer!





Yatzy II

- Kursen introducerade slump genom början på ett Yatzy-spel i föreläsning #2, det blev inte mycket mer än så här:

```
Yatzy 0.1
[ 2 2 6 3 5 ]
Done, bye!
```

- Vad behöver vi göra om vi ska gå vidare?

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define DICE 6
#define HAND 5

//*** Random Number Package *************** 1.0/AC ***
void initRandomGenerator(void){
    srand((unsigned)time(NULL));
}
int rndNumber(int nbs){
    return rand()%nbs;
}
//*** RNP END ****************************************

int main(int argc, char **argv)
{
    printf("Yatzy 0.1\n");

    printf("[ ");
    for (int i=0; i<HAND; i++)
        printf("%d ",rndNumber(DICE)+1);
    printf("]\n");

    printf("Done, bye!\n");
    return 0;
}
```



Yatzy II

- Kursen introducerade slump genom början på ett Yatzy-spel i föreläsning #2, det blev inte mycket mer än så här:

```
Yatzy 0.1
[ 2 2 6 3 5 ]
Done, bye!
```

- Vad behöver vi göra om vi ska gå vidare?
  - **Kasten måste sparas!**
  - **Funktion för att skriva ut!**

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define DICE 6
#define HAND 5

//*** Random Number Package *************** 1.0/AC ***
void initRandomGenerator(void){
    srand((unsigned)time(NULL));
}
int rndNumber(int nbs){
    return rand()%nbs;
}
//*** RNP END ****************************************

int main(int argc, char **argv)
{
    printf("Yatzy 0.1\n");

    printf("[ ");
    for (int i=0; i<HAND; i++)
        printf("%d ",rndNumber(DICE)+1);
    printf("]\n");

    printf("Done, bye!\n");
    return 0;
}
```

Yatzy II

- Kursen introducerade slump genom början på ett Yatzy-spel i föreläsning #2, det blev inte mycket mer än så här:

```
Yatzy 0.2
[ 2 5 6 6 1 ]
Done, bye!
```

- Vad behöver vi göra om vi ska gå vidare?
  - **Kasten måste sparas!**
  - **Funktion för att skriva ut!**
  - **Slå om vissa tärningar?!**

```
//*** Yatzy Game Package ******************** 0.2/AC ***
     viewHand(                                  ){

}
//*** YGP END ****************************************
int main(int argc, char **argv)
{
    int hand[HAND];
    printf("Yatzy 0.2\n");
    initRandomGenerator();

    for (int i=0; i<HAND; i++)
        hand[i]=rndNumber(DICE)+1;
    viewHand(hand,HAND);

    printf("Done, bye!\n");
    return 0;
}
```

Yatzy II

- Kursen introducerade slump genom början på ett Yatzy-spel i föreläsning #2, det blev inte mycket mer än så här:

```
Yatzy 0.2
[ 2 5 6 6 1 ]
Done, bye!
```

- Vad behöver vi göra om vi ska gå vidare?
  - **Kasten måste sparas!**
  - **Funktion för att skriva ut!**
  - **Slå om vissa tärningar?!**

```
//*** Yatzy Game Package ******************** 0.2/AC ***
void viewHand(int aHand[], int size){
    printf("[ ");
    for (int i=0; i<size; i++)
        printf("%d ",aHand[i]);
    printf("]\n");
}
//*** YGP END ****************************************
int main(int argc, char **argv)
{
    int hand[HAND];
    printf("Yatzy 0.2\n");
    initRandomGenerator();

    for (int i=0; i<HAND; i++)
        hand[i]=rndNumber(DICE)+1;
    viewHand(hand,HAND);

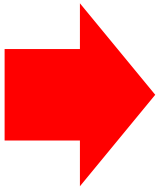
    printf("Done, bye!\n");
    return 0;
}
```



Yatzy II

- Kursen introducerade slump genom början på ett Yatzy-spel i föreläsning #2, det blev inte mycket mer än så här:

```
Yatzy 0.2
[ 2 5 6 6 1 ]
Done, bye!
```



```
Yatzy 0.3
[ 3 6 1 5 1 ]
Roll dice (i.e. 23) : 124
[ 4 4 1 2 1 ]
Roll dice (i.e. 23) : 345
[ 4 4 4 2 4 ]
Done, bye!
```

- Vad behöver vi göra om vi ska gå vidare?
  - **Kasten måste sparas!**
  - **Funktion för att skriva ut!**
  - **Slå om vissa tärningar?!**

Siffror eller bokstäver & return?!

Yatzy II

- Kursen introducerade slump genom början på ett Yatzy-spel i föreläsning #2, det blev inte mycket mer än så här:

```
Yatzy 0.3
[ 3 6 1 5 1 ]
Roll dice (i.e. 23) : 124
[ 4 4 1 2 1 ]
Roll dice (i.e. 23) : 345
[ 4 4 4 2 4 ]
Done, bye!
```

- Vad behöver vi göra om vi ska gå vidare?
  - **Kasten måste sparas!**
  - **Funktion för att skriva ut!**
  - **Slå om vissa tärningar?!**

```
int main(int argc, char **argv)
{
    int hand[HAND];
    char roll;
    printf("Yatzy 0.3\n");
    initRandomGenerator();

    for (int i=0; i<HAND; i++)
        hand[i]=rndNumber(DICE)+1;
    viewHand(hand,HAND);

    for (int t=0; t<2; t++) {
        printf("Roll dice (i.e. 23) : ");

        viewHand(hand,HAND);
    }

    printf("Done, bye!\n");
    return 0;
}
```

‘1’ ‘2’ ‘4’ ‘\n’

Yatzy II

- Kursen introducerade slump genom början på ett Yatzy-spel i föreläsning #2, det blev inte mycket mer än så här:

```
Yatzy 0.3
[ 3 6 1 5 1 ]
Roll dice (i.e. 23) : 124
[ 4 4 1 2 1 ]
Roll dice (i.e. 23) : 345
[ 4 4 4 2 4 ]
Done, bye!
```

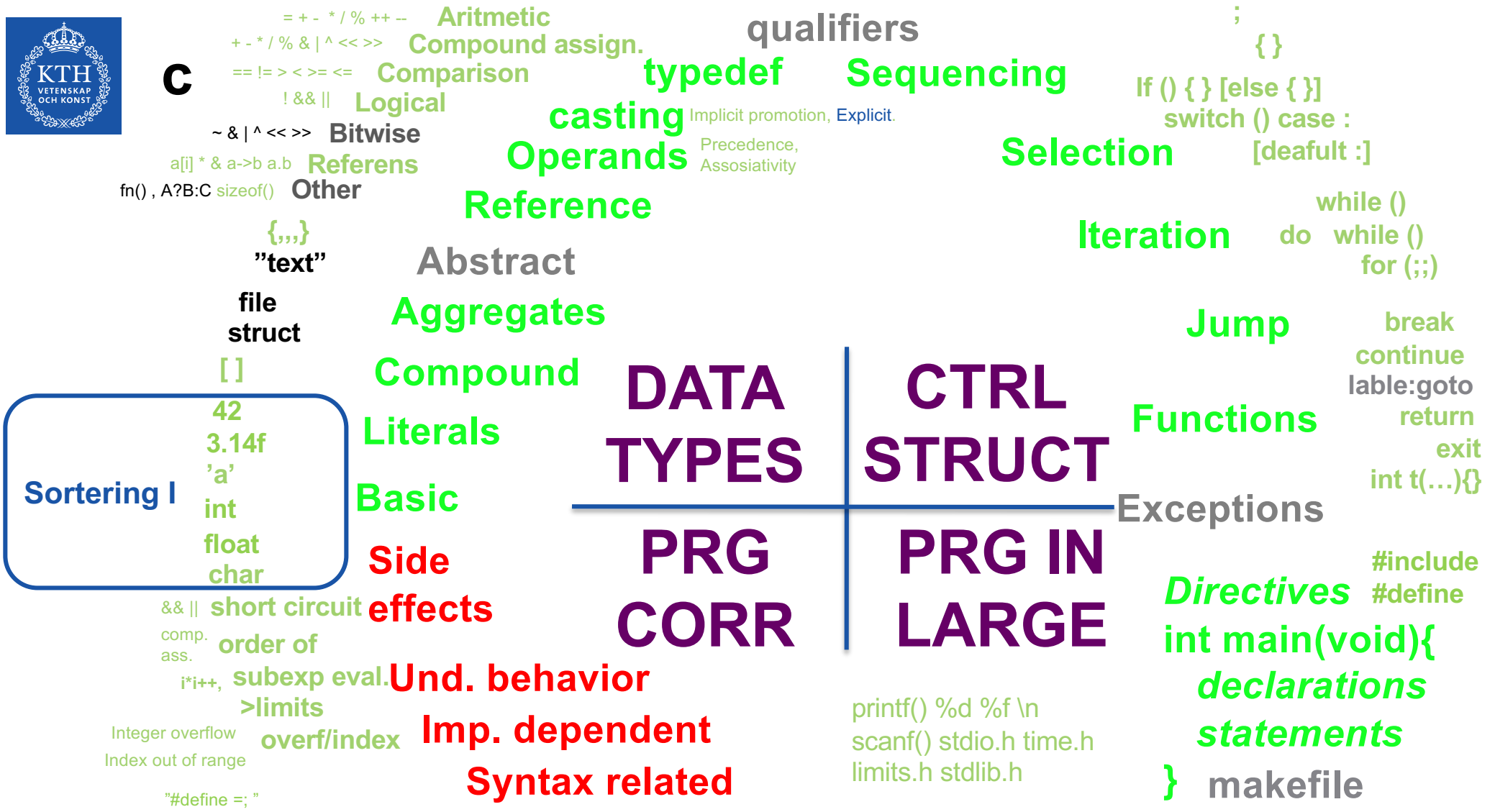
- Vad behöver vi göra om vi ska gå vidare?
  - **Kasten måste sparas!**
  - **Funktion för att skriva ut!**
  - **Slå om vissa tärningar?!**

```
int main(int argc, char **argv)
{
    int hand[HAND];
    char roll;
    printf("Yatzy 0.3\n");
    initRandomGenerator();

    for (int i=0; i<HAND; i++)
        hand[i]=rndNumber(DICE)+1;
    viewHand(hand,HAND);

    for (int t=0; t<2; t++) {
        printf("Roll dice (i.e. 23) : ");
        do {
            scanf("%c",&roll);
If (roll!='\n') hand[roll-'1']=rndNumber(DICE)+1;
        } while (roll!='\n');
        viewHand(hand,HAND);
    }

    printf("Done, bye!\n");
    return 0;
}
```


C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
Aggregates
Compound
Literals
Basic
Sortering I
42
3.14f
’a’
int
float
char
Abstract
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence,
Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
Iteration
do while ()
for (;;)
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
}
makefile
&& || short circuit
comp. ass.
order of
i*i++,
subexp eval.
>limits
Integer overflow
Index out of range
overf/index
Und. behavior
Imp. dependent
Syntax related
”#define =; ”
printf() %d %f \n
scanf() stdio.h time.h
limits.h stdlib.h
int/int
/*comment*/ or //

Övn #10: Lab #2

Att använda datorprogram för mätdatainsamling och analys är mycket vanligt. Ni ska skriva ett välstrukturerat program där man kan mata in mätvärden och analysera dessa. När programmet startar får användaren upp en meny där denne kan välja:

v (View) - visar vilka mätvärden som finns lagrade
e (Enter) - tillåter användaren att lägga till mätvärden
c (Compute) - presenterar max, min, medelvärde och normaliserade mätvärden
r (Reset) - raderar alla inmatade mätvärden
q (Quit) - avslutar programmet

Användaren ska göra sitt val genom att ange en bokstav…

Hela uppgiften finns på Canvas!

```
Measurement tool 1.2
VECRQ? r
VECRQ? v
[ No measurements ]
VECRQ? e
Enter measurement #1 (or 0) ?1
Enter measurement #2 (or 0) ?2
Enter measurement #3 (or 0) ?3
Enter measurement #4 (or 0) ?0
VECRQ? v
[ 1 2 3 ]
VECRQ? e
Enter measurement #4 (or 0) ?4
Enter measurement #5 (or 0) ?5
Enter measurement #6 (or 0) ?0
VECRQ? v
[ 1 2 3 4 5 ]
VECRQ? c
Max value : 5
Min value : 1
Avr value : 3.00
[ -2 -1 0 1 2 ]
VECRQ? q
Exit Measurement tool!
```



Övn #10: 11: E5

```
split_time(long total_sec,
           int *hr,
           int *min,
           int *sec);
```

1 Skriv ner proble-met så att ett barn förstår det!

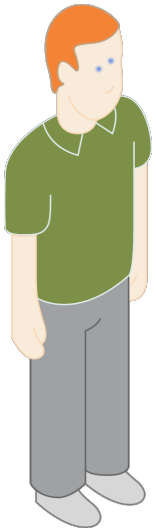
2 Gör ett minimalt main() av dina anteckningar!

3 Konstanter Variabler med Värde Uttryck & resultat!

a..n

4 Läs in riktiga värden!

Historik



• 150726 0.1	AC	Skapad.
• 150727 0.5	AC	Sortering.
• 150728 1.0	AC	Övning.
• 190801 2.0	AC	Omarbetad från grunden, samt anpassad till KTH:s nya profil.